

Guía del sistema: cómo está organizado, qué hace cada módulo y por qué están diseñados así.

1. Visión general

MDP-ProbLog permite especificar un Proceso de Decisión de Markov (MDP) en sintaxis Prolog: el usuario escribe hechos y reglas que describen estados, acciones, probabilidades de transición y recompensas. El sistema lee esa especificación, la convierte en un problema de inferencia probabilística y resuelve el MDP calculando la función de valor óptima y la política greedy mediante Value Iteration.

2. Estructura del proyecto

```
mdp-problog/  
├─ engine.py           # Adaptador sobre ProbLog (parseo, inyección,  
│                       compilación, evaluación)  
├─ mdp.py              # Clase MDP: puente central entre el modelo y el  
solver  
├─ value_iteration.py  # Value Iteration con backups de Bellman  
└─ fluent/  
    ├─ __init__.py      # Subpaquete para el manejo de fluentes de estado  
    ├─ schema.py        # Fluent, FluentSchema: estructuras de datos del  
    │                   espacio de estados  
    ├─ spaces.py        # FactorSpace, StateSpace, ActionSpace: iteración  
    │                   e indexación  
    ├─ classification.py # FluentClassifier: Clasificación de fluentes  
    └─ exceptions.py    # Jerarquía de excepciones
```

3. Engine

La clase `Engine` envuelve el motor de inferencia de ProbLog y expone las operaciones que MDP-ProbLog necesita. Internamente mantiene tres representaciones sucesivas del programa, que corresponden a las etapas de compilación:

Atributo	Descripción
<code>_db</code>	Tabla de instrucciones: contiene el programa original más todas las inyecciones de hechos dummy
<code>_gp</code>	Programa proposicional aterrizado: solo las cláusulas relevantes para las consultas declaradas
<code>_knowledge</code>	Circuito lógico compilado: permite evaluación de probabilidades en tiempo polinomial

Consulta de declaraciones

Operación	Descripción
<code>declarations(type)</code>	Lista todos los términos declarados con el predicado <code>type/1</code>
<code>assignments(type)</code>	Diccionario de pares clave-valor del predicado <code>type/2</code>

Inyección en la tabla de instrucciones

Operación	Efecto
<code>add_fact(term, probability)</code>	Inserta un hecho probabilístico
<code>add_rule(head, body)</code>	Inserta una regla lógica <code>head :- body</code>
<code>add_assignment(term, value)</code>	Inserta un hecho <code>utility(term, value)</code>
<code>add_annotated_disjunction(facts, probs)</code>	Inserta un grupo de hechos mutuamente excluyentes con probabilidades dadas

Compilación y evaluación

Operación	Descripción
<code>relevant_ground(queries)</code>	Aterrizar el programa respecto a la lista de términos de consulta
<code>compile(terms)</code>	Compila el programa aterrizado y mapea cada término a su nodo en el circuito
<code>evaluate(queries, evidence)</code>	Calcula probabilidades marginales dado un conjunto de evidencia

`get_ads_inverted_index()`

Recorre la tabla de instrucciones buscando nodos `choice` (nodos internos de las Disyunciones Anotadas de ProbLog). Para cada nodo extrae el término que representa y registra sus argumentos en un índice invertido:

```
{ valor_string → { group_id_1, group_id_2, ... } }
```

Este índice es usado por `FluentClassifier` para determinar si los valores de un fluente implícito se originan en una Disyunción Anotada (y por tanto el fluente es de tipo `multivalued`).

4. Fuentes (`fluent/`)

4.1 Estructuras de datos (`schema.py`)

`Fluent`

Clase encargada de la tarea única de crear términos que representen un fluente:

`Fluent.create_fluent(term, timestep)` — devuelve una copia del término con `timestep` añadido como último argumento. Los fluentes son atemporales en el schema; sus versiones temporales se producen bajo demanda con esta operación.

- Estado actual (evidencia): `term(args..., 0)`
- Siguierte estado (consulta): `term(args..., 1)`

`FluentSchema`

Define la estructura factorizada del espacio de estados. Mantiene una lista ordenada de factores, donde cada factor es:

- **Bool** (base 2): una variable binaria independiente que toma valores en {0, 1}.
- **Multivalued** (base N): un grupo de N opciones mutuamente excluyentes donde exactamente una está activa (codificación one-hot).

Operación	Descripción
<code>add_bool(term)</code>	Registra un fluente booleano como nuevo factor de base 2
<code>add_group(terms)</code>	Registra un grupo multivaluado como nuevo factor de base N

Operación	Descripción
<code>get_local_index(factor_index, temporal_term)</code>	Devuelve la posición del término dentro de su factor (0 para la rama False de un bool)

La propiedad `strides` devuelve la lista de desplazamientos posicionales usados para la indexación. La propiedad `total_states` devuelve el producto de todas las bases.

Indexación de raíz mixta

El espacio de estados de un MDP factorizado es el **producto cartesiano** de factores independientes. Si el dominio tiene tres factores con bases $b_0 = 2$, $b_1 = 3$, $b_2 = 2$, el espacio total tiene $2 \times 3 \times 2 = 12$ estados distintos.

Para identificar cada estado con un número entero único se usa el mismo principio que los **sistemas de numeración posicional**, pero con bases distintas por posición. Realizo este proceso ya que cada configuración de estado está listada y almacenada en un array unidimensional y es necesario saber la posición exacta de una configuración de estado determinada y vice-versa.

Strides. El *stride* del factor k es el producto de todas las bases anteriores:

$$s_k = \prod_{j < k} b_j$$

Para bases $[2, 3, 2]$ los strides son $[1, 2, 6]$.

Codificación (valuación $\rightarrow$ índice entero):

$$\text{idx} = \sum_k l_k \cdot s_k$$

donde l_k es el índice local del valor activo en el factor k (0 ó 1 para un fluente booleano; 0, 1, ..., N-1 para un fluente multivaluado).

Descomposición (índice entero $\rightarrow$ valuación):

$$l_k = \text{idx} \bmod b_k, \quad \text{idx} \leftarrow \left\lfloor \frac{\text{idx}}{b_k} \right\rfloor$$

Se procesa cada factor de izquierda a derecha. El residuo módulo de la base, da el índice local; la división entera prepara el índice para el siguiente factor.

Ejemplo concreto. Bases $[2, 3, 2]$, strides $[1, 2, 6]$, espacio de 12 estados:

La siguiente tabla contiene todo el espacio de estados. Cada entrada representa una configuración de estados dada por la distribución de valores activos de cada factor.

Índice	Factor 0 ($b_0 = 2$)	Factor 1 ($b_1 = 3$)	Factor 2 ($b_2 = 2$)
0	0	0	0
1	1	0	0
2	0	1	0
3	1	1	0
4	0	2	0
5	1	2	0
6	0	0	1
7	1	0	1
8	0	1	1
9	1	1	1
10	0	2	1
11	1	2	1

Para **codificar** el estado 5, tenemos que obtener los valores de cada uno de los factores.

En este caso tenemos que los índices de los valores activos para cada factor b_k son :

1. Factor 0 ($b_0 = 2$) = 1;
2. Factor 1 ($b_1 = 3$)= 2;
3. Factor 2 ($b_2 = 2$)= 0.

Y los strides correspondientes a cada factor:

1. Factor 0 ($s_0 = 2$) = 1;
2. Factor 1 ($s_1 = 2$)= 2;
3. Factor 2 ($s_2 = 2$)= 0.

La operación realizada sería:

$$(1 \cdot 1) + (2 \cdot 2) + (0 \cdot 6) = 5$$

Para **decodificar** el índice 9:

1. $9 \bmod 2 = 1$ (factor 0), $\lfloor 9/2 \rfloor = 4$;
2. $4 \bmod 3 = 1$ (factor 1), $\lfloor 4/3 \rfloor = 1$;

3. $1 \bmod 2 = 1$ (factor 2).

Resultado:

	Factor 0	Factor 1	Factor 2
Indice del valor activo	1	1	1

4.2 Clasificación de fluentes (`classification.py`)

`FluentClassifier` orquesta la extracción, validación e inferencia de los fluentes de estado declarados en el programa Prolog. Su operación principal es `classify()`, que retorna un `FluentSchema` validado.

El flujo se resume en tres fases:

Fase 1 — Validación estática (`_validate_fluent_declarations`): verifica que las etiquetas de tipo en las declaraciones explícitas sean válidas (`bool` o `multivalued`). Si hay duplicados entre declaraciones implícitas y explícitas, emite una advertencia indicando que la declaración explícita tiene precedencia.

Fase 2 — Registro y clasificación:

- `_register_explicit`: parsea los fluentes declarados con `state_fluent/2` y lee su tipo directamente de la etiqueta.
- `_register_implicit`: agrupa los fluentes declarados con `state_fluent/1` por predicado y llama a `_infer_fluent_type`, que consulta el índice invertido de Disyunciones Anotadas para determinar si el flujo es `multivalued` (sus valores se originan en una AD) o `bool` (en caso contrario).

Fase 3 — Distribución y construcción:

- `_separte_mv_fluents`: envía los fluentes booleanos directamente a `schema.add_bool()` y acumula los multivaluados agrupados por functor.
- `_validate_multivalued`: valida que cada grupo multivaluado tenga al menos 2 opciones, luego los registra en el schema con `schema.add_group()`. Los errores de cardinalidad se acumulan y se lanzan agrupados.

4.3 Espacios de iteración (`spaces.py`)

`FactorSpace` es la clase base que implementa la codificación y decodificación de raíz mixta para iterar sobre valuaciones. `StateSpace` y `ActionSpace` son subclases especializadas.

StateSpace itera sobre todos los estados del MDP. Cada elemento es un `OrderedDict` que mapea términos temporales (a timestep 0 por defecto) a sus valores enteros (0 ó 1 para bool; codificación one-hot para multivalued).

ActionSpace trata las acciones como un único grupo mutuamente excluyente. Los términos de acción no llevan timestep.

Ambas soportan:

- `__getitem__(i)` : decodifica el índice i en una valuación usando la descomposición módulo/división por factor.
- `index(valuation)` : codifica una valuación en su índice entero usando la suma de índices locales multiplicados por sus strides.

5. MDP (`mdp.py`)

MDP es el puente central entre el programa Prolog del usuario y el algoritmo de resolución. El constructor recibe el programa como string y ejecuta `__prepare()`.

```
MDP(model, epsilon_thr=1e-6, backend=None)
```

5.1 Preparación (`__prepare`)

El método ejecuta cinco etapas en orden:

Etap 1 — Clasificación: delega al `FluentClassifier` la extracción e inferencia de todos los fluentes. El resultado es el `state_schema` que describe el espacio de estados factorizado.

Etap 2 — Inyección de hechos dummy (t=0): para que el evaluador de ProbLog pueda calcular $P(\text{siguiente estado} \mid \text{estado actual, acción})$, el estado actual y la acción deben poder fijarse como evidencia. Esto requiere que existan en la tabla de instrucciones como variables probabilísticas con peso sustituible. Por eso se inyectan hechos "dummy" antes del grounding:

- **Bool:** `add_fact(fluent(args, 0), 0.5)` . La probabilidad 0.5 es neutral; será reemplazada por 1.0 (verdadero) o 0.0 (falso) al pasar evidencia.
- **Multivalued:** `add_annotated_disjunction([opt_1(0), opt_2(0), ...], [1/N, ...])` . Las probabilidades uniformes son también neutrales; la evidencia forzará una de las opciones a 1.0.

Etapas 3 — Inyección de acciones: las acciones se obtienen de `declarations('action')` y se inyectan como una única Disyunción Anotada con probabilidades uniformes, de forma análoga a los fluentes multivaluados.

Etapas 4 — Grounding relevante: se construye la lista de consultas que incluye utilidades, fluentes $t=1$, acciones y fluentes $t=0$. Incluir los fluentes $t=0$ es imprescindible: garantiza que ProbLog preserve sus nombres en el programa aterrizado, de modo que el evaluador pueda localizarlos al sustituir evidencia.

Etapas 5 — Compilación: los fluentes $t=1$ y las utilidades se compilan en nodos del circuito lógico. El resultado son dos mapas `term → nodo` que se reutilizan en cada evaluación.

5.2 Evaluación de transiciones y recompensas

Operación	Entrada	Salida
<code>transition(state, action, cache)</code>	Valuación del estado actual y la acción	Lista plana [(fluente_t1, prob), ...]
<code>structured_transition(state, action, cache)</code>	Ídem	Lista de factores: [(term\ None, prob), ...], ...]
<code>reward(state, action, cache)</code>	Ídem	Recompensa inmediata esperada como float

`transition()` evalúa el circuito compilado pasando la unión del estado y la acción como evidencia. `structured_transition()` llama a `transition()` y agrupa el resultado por factores del schema.

El parámetro `cache` es una clave opcional (típicamente la tupla `(state_index, action_index)`). Cuando se provee, el resultado se almacena en un diccionario interno y no se re-evalúa el circuito en llamadas posteriores con la misma clave.

Filtrado de ramas poco probables

`structured_transition()` aplica un filtro sobre cada factor antes de devolver el resultado. El umbral es `epsilon_thr` (1e-6 por defecto, configurable en el constructor de `MDP`).

Para **factores booleanos**: se calcula $P(\text{true})$ directamente del mapa de probabilidades y $P(\text{false}) = 1 - P(\text{true})$. Solo se incluye una rama en el resultado si su probabilidad es estrictamente mayor que `epsilon_thr`.

Para **factores multivaluados**: se evalúa la probabilidad de cada opción del grupo individualmente. Solo se incluyen las opciones con probabilidad estrictamente mayor que `epsilon_thr`.

Por qué se hace?. En los MDPs, la mayoría de las transiciones tienen probabilidad prácticamente nula bajo ciertos pares (estado, acción). Por ejemplo, si una regla de transición nunca se activa con una acción particular, todas las opciones de ese factor excepto una tendrán probabilidad cercana a cero. Incluirlas en el cálculo del valor esperado no cambia el resultado numéricamente, pero sí multiplica innecesariamente el costo de la recursión sobre el árbol de transiciones factorizado. El umbral $1e-6$ delimita lo irrelevante.

Consecuencia práctica. El número de ramas activas por factor es siempre menor o igual a la base del factor; en la práctica suele ser 1 (transición determinista) o ≥ 2 (transición estocástica binaria). Esto hace que la recursión de Value Iteration recorra un árbol muy pequeño en lugar de enumerar todos los estados destino.

6. Value Iteration (`value_iteration.py`)

6.1 Algoritmo

`ValueIteration` ejecuta backups de Bellman síncronos sobre todos los estados hasta convergencia:

```
Inicializar  $V(s) = 0$  para todo  $s$ 

Repetir:
    Para cada estado  $s$  (índice  $i$ ):
        Para cada acción  $a$  (índice  $j$ ):
             $T \leftarrow \text{mdp.structured\_transition}(s, a, \text{cache}=(i,j))$ 
             $R \leftarrow \text{mdp.reward}(s, a, \text{cache}=(i,j))$ 
             $Q(s, a) \leftarrow R + \gamma \cdot \text{Expected\_Val}(T)$ 
         $V(s) \leftarrow \max_a Q(s, a)$ 
         $\pi(s) \leftarrow \text{argmax}_a Q(s, a)$ 

     $\text{max\_residual} \leftarrow \max_s |V_{\text{nuevo}}(s) - V_{\text{anterior}}(s)|$ 

Hasta que  $\text{max\_residual} \leq 2\epsilon(1 - \gamma) / \gamma$ 
```

Criterio de convergencia. El umbral $2\epsilon(1 - \gamma)/\gamma$ garantiza que la política greedy resultante es ϵ -óptima respecto a la función de valor real del MDP descontado.

6.2 Cálculo del valor esperado (`__expected_value`)

En lugar de sumar $V(s')$ sobre todos los estados destino posibles por par estado-acción, la recursión aprovecha la estructura factorizada de las transiciones:

```
def __expected_value(self, transition_groups, strides, V, k=0,
current_index=0, joint=1.0):
    if len(transition_groups) == k:
        return joint * V.get(current_index, 0.0)

    factor = transition_groups[k]
    stride = strides[k]
    expected_sum = 0.0

    for term, prob in factor:
        val = self._mdp.state_schema.get_local_index(k, term)
        expected_sum += self.__expected_value(
            transition_groups, strides, V,
            k + 1, current_index + val * stride, joint * prob
        )
    return expected_sum
```

En cada nivel k de la recursión se toman las ramas activas del factor k (post-filtrado). Para cada rama:

1. Se obtiene el índice local l_k del término con `get_local_index`.
2. Se acumula al índice global: $\text{current_index} + l_k \cdot s_k$.
3. Se multiplica la probabilidad conjunta: $\text{joint} \cdot p$.

Al llegar al caso base ($k = \text{\$número de factores}$), el índice acumulado identifica unívocamente el estado destino y se retorna $\text{\$joint}$.

El resultado es equivalente a $\sum_{s'} P(s' | s, a) \cdot V(s')$, pero calculado solo sobre las ramas activas de cada factor sin enumerar el espacio completo de estados.

7. Invariantes del sistema

- **Determinismo del schema.** Los fluentes se registran en orden alfabético (`sorted`), garantizando que el schema sea idéntico entre ejecuciones con el mismo programa.
- **Caché por índice.** Las claves de caché (i, j) corresponden a pares (estado, acción), evitando colisiones y garantizando que cada par se evalúa como máximo una vez por sesión.

- **Preservación de nombres.** Los flujos $t=0$ se incluyen en las consultas de grounding para que ProbLog conserve sus identificadores en el programa aterrizado, lo que permite localizarlos al sustituir evidencia durante la evaluación.
- **Cardinalidad mínima.** Todo grupo multivaluado tiene al menos 2 opciones; `_finalize_multivalued` lanza `FluentCardinalityError` si se viola esta condición.